

03 — Documentation technique

Choix technologiques, versions, dépendances, modèles IA · Juillet 2026

SANAD — Documentation technique : choix technologiques, versions, dépendances

1. Choix technologiques et justifications

Besoin	Choix	Justification
Frontend + API gateway	Next.js 16 (App Router, React 19, TypeScript 5)	Un seul framework pour l'UI (client + assureur) et les routes API serveur (secrets côté serveur, pas d'API key exposée au navigateur) ; middleware natif pour l'authentification par rôle
OCR carte d'identité	PaddleOCR PP-OCRv5 (lang= 'ar')	Meilleure précision arabe/latin mixte que Tesseract sur les CIN tunisiennes ; tourne 100 % en local (les pièces d'identité ne quittent pas le serveur)
OCR documents sinistre	Tesseract.js (fra+ara+eng) + pdf-parse	Exécution dans le runtime Node sans service supplémentaire ; suffisant pour factures/rapports dactylographiés
Face-match / liveness	OpenCV (Haar cascade, NCC, variance Laplacienne)	Léger, local, sans GPU ; anti-spoofing basique (photo de photo) sans dépendance cloud
LLM (FNOL, extraction, vision)	Groq API (mode JSON strict)	Latence très faible (chat temps réel), sortie JSON contrainte pour un intake structuré fiable ; support multilingue FR/AR/derja
Détection dégâts véhicule	Roboflow (YOLO car-detection-damage/3)	Modèle spécialisé entraîné sur des dégâts automobiles ; API serverless simple ; fallback Groq vision puis heuristique déclarative → le flux ne bloque jamais
Base de données	MongoDB + Mongoose 9	Schéma flexible pour les rapports de sinistre (structure évolutive), index sur cin / userId
Authentification	jose (JWT) + bcryptjs	Cookies httpOnly signés, vérifiables dans le middleware Edge ; hachage de mots de passe éprouvé
PDF (constat, rapport police)	pdf-lib	Génération client/serveur sans binaire externe
Audit trail KYC	Fichiers JSON (1/dossier)	Simplicité, lisibilité, portabilité inter-assureurs ; migration triviale vers une base documentaire
AML/CFT	Moteur Python maison (fuzzy matching diff1ib)	Transparence totale du scoring (exigence d'explicabilité réglementaire, Loi 2015-26) ; listes remplaçables par les sources officielles

2. Versions

Composant	Version
Node.js	≥ 20
Next.js / React / TypeScript	16.2.10 / 19.2.4 / ^5
Python	3.10 – 3.11
FastAPI / Uvicorn	0.110.0 / 0.28.0
PaddleOCR / PaddlePaddle	≥ 2.7.3 / ≥ 2.5.2
OpenCV (headless)	4.9.0.80
MongoDB / Mongoose	≥ 6 / 9.7.4
Tesseract.js	7.0.0
Vitest (tests TS)	^4

3. Dépendances principales

- Node (package.json)** : next, react, react-dom, mongoose, jose, bcryptjs, tesseract.js, pdf-lib, pdf-parse, recharts, @napi-rs/canvas (redimensionnement images vision), lucide-react, @heroicons/react, tailwindcss 4.
- Python (ai-services/ocr-service/requirements.txt)** : fastapi, uvicorn, paddleocr, paddlepaddle, python-multipart, opencv-python-headless, pillow, pydantic 2, httpx, numpy.
- Services externes** : Groq API (LLM texte + vision), Roboflow Serverless (YOLO), SMTP (reçus KYC).

4. Modèles d'IA utilisés

Tâche	Modèle	Où
OCR CIN	PP-OCRv5 arabe (détection + reconnaissance + orientation de ligne)	Local (service Python)
Chat FNOL + extraction documents	openai/gpt-oss-120b via Groq (surchargeable GROQ_MODEL)	Cloud
Vision (dégâts fallback, anti-fraude)	qwen/qwen3.6-27b via Groq (surchargeable GROQ_VISION_MODEL)	Cloud
Détection dégâts	YOLO car-detection-damage/3	Roboflow serverless
OCR documents sinistre	Tesseract LSTM fra+ara+eng	Local (Node)

5. Décisions d'architecture notables

- Microservice eKYC séparé** : l'OCR lourd (PaddleOCR) et les données biométriques restent isolés dans un service Python dédié — scalable et déployable indépendamment (Dockerfile fourni).

- **Chaîne de fallback systématique** pour l'estimation des dégâts (Roboflow → Groq vision → heuristique) et pour l'email (SMTP → simulation loggée) : aucune panne d'un fournisseur ne bloque le parcours utilisateur.
- **Double vérification anti-doublon CIN** (MongoDB + audit Python) : la fraude à l'identité est détectée même si l'un des deux stores est incomplet.
- **Explicabilité intégrée** : chaque décision (score AML, score de fraude, fourchette d'indemnisation, routage auto/manuel) est accompagnée de facteurs pondérés lisibles — pas de « boîte noire ».
- **Stateless côté chat** : l'historique FNOL est rejoué à chaque tour ; pas d'état serveur à gérer, reprise de session triviale.

6. Limites connues / axes d'amélioration

- Le dashboard assureur affiche les sinistres depuis des données de démonstration (`src/data/mockClaims.ts`) ; l'endpoint réel `GET /api/claims` existe et est testé — le branchement UI est un chantier court.
- Le seuil AML `CRITICAL` bloque l'inscription côté client ; une revalidation serveur dans `/api/auth/register` renforcerait la garantie.
- `parse_identity_card` injecte des valeurs par défaut si l'extraction échoue (mode démo) — à désactiver en production.
- PDF scannés (image pure) non rasterisés côté extraction de documents (avertissement retourné).
- Les listes sanctions/PPE sont synthétiques (démo) ; production = listes officielles ONU/CTAF avec mise à jour périodique.